

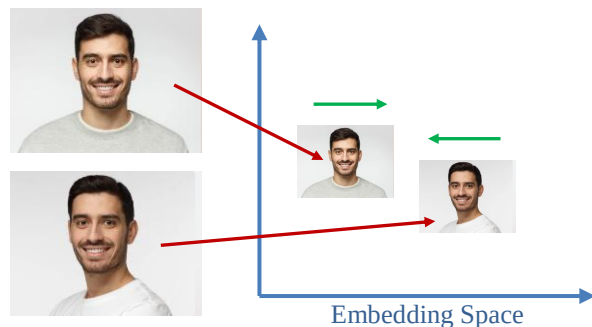
Embeddings in Deep Learning

Embeddings in Deep Learning

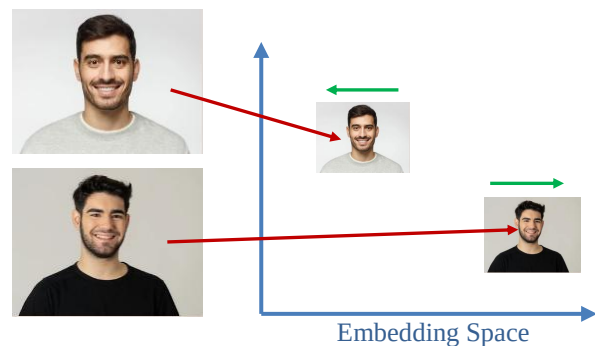
- An **embedding** is a learned mapping of input data from a high-dimensional space to a lower-dimensional vector representation.
 - These can be viewed as feature vectors learned automatically by the model.
 - These vectors lie in an **embedding space**, which is a geometric space where relationships between data points are preserved.
 - In this space distance reflects similarity; similar inputs are mapped to nearby points, while dissimilar inputs are placed farther apart.
 - Embeddings capture meaningful patterns and relationships, allowing models to process data more efficiently.
 - They are learned during training and can often be reused across different models and tasks.
- Types of Embeddings
- **Word Embeddings:** Represent words as real-valued vectors in an embedding space, where words with similar meanings are located close to each other.
 - **Visual Embeddings:** Represent images as low-dimensional vectors in an embedding space, enabling tasks such as image comparison, similarity search, and recognition.

Visual Embeddings

- A **visual embedding** is a low-dimensional vector representation of an image.
- These vectors lie in an embedding space, where the position of each image reflects its meaning.
- In this space similar images are close together and dissimilar images are far apart. Thus, images can be compared by measuring the distance between their embeddings.



Similar Images



Dissimilar Images

- We need embeddings as direct comparison of images using raw pixels is not effective because:
 - Images are high-dimensional, making computation expensive
 - Pixel values do not capture semantic similarity
 - Natural images have complex structure and variation
- Embeddings address this by:
 - Providing a compact representation
 - Capturing meaningful features for comparison

One-shot Learning

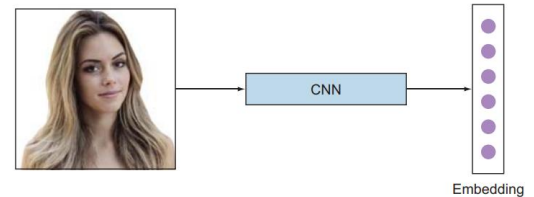
- Learning embeddings enables similarity-based comparison, which makes one-shot learning possible.
- One-shot learning is a learning setting in which a model must correctly recognize or classify new classes using only one (or very few) labeled examples per class.
- Applications: face recognition, image recommendation, object re-identification, signature verification, and fingerprint matching.
- Face recognition relies on comparing embeddings of face images. Two main modes exist:
 - Face Verification (1:1): Compare two images to determine whether they belong to the same person.
 - Face Identification (1:many): Compare a query image with a database to determine the person's identity.

System sees 1 liked image Learns its embedding/style

Recommends similar images (sunset, beach, warm tones) : image recommendation

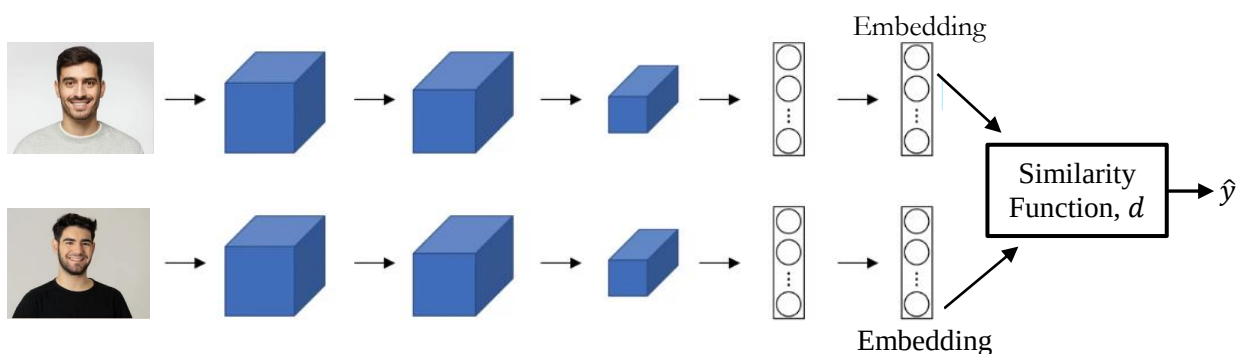
Learning Visual Embeddings

- Visual embeddings are typically learned using a Convolutional Neural Network (CNN), where the output of the final layer (before the loss function) is used as the embedding.
- The network is trained so that similar images are mapped close together, and dissimilar images are mapped far apart.
- This is achieved using similarity-based loss functions.
- After training, the network can convert any image into its embedding, and similarity is computed using distance in embedding space.



Siamese Network

- A Siamese network is a neural architecture designed for learning similarity between inputs.
- It consists of two identical subnetworks that:
 - Share the same architecture
 - Share the same weights and parameters
- Each subnetwork processes one input and produces an embedding. The similarity (or distance) between the two embeddings is then computed.
- The objective is to determine whether two inputs are similar ($y = 1$) or dissimilar ($y = 0$). This is typically done using a distance metric, such as Euclidean distance or cosine similarity.
- Instead of directly applying a standard log-loss with a threshold, Siamese networks are usually trained using contrastive loss or triplet loss, which explicitly encourage small distance for similar inputs, and large distance for dissimilar inputs.



- The training data consists of pairs of inputs:
 - Positive pair ($y = 1$): Both inputs belong to the same class
 - Negative pair ($y = 0$): Inputs belong to different classes
- The network learns to:
 - Minimize distance for positive pairs
 - Maximize distance for negative pairs

